



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارش پیشرفت پروژه اینترنت اشیا - هفته سوم

IOT Project Progress Report - Week 3

اعضای گروه

علی فتحی

آرمین فارسی

امیررضا بهرامی

نام درس

اینترنت اشیا

تابستان ۱۴۰۵

نام استاد درس

دکتر اجلالی

طراحی و پیاده‌سازی سامانه مدیریت OTA برای ESP32

در هفته سوم، تمرکز اصلی پروژه بر افزایش اطمینان‌پذیری فرایند OTA قرار گرفت. در این مرحله، بررسی صحت فایل ثابت‌افزار با استفاده از SHA-256 در سمت بورد تکمیل شد. سپس سازوکار Rollback برای بازگشت خودکار به نسخه قبلی، در صورت ناموفق بودن اجرای نسخه جدید، پیاده‌سازی و آزمایش شد. در پایان نیز چرخه کامل دریافت نسخه، بررسی صحت، نصب، تشخیص خطا و بازگشت به نسخه سالم قبلی با موفقیت ارزیابی شد.

۱. بررسی صحت فایل ثابت‌افزار با SHA-256

در مراحل قبل، مقدار SHA-256 فایل ثابت‌افزار در سمت سرور محاسبه و همراه اطلاعات نسخه جدید برای بورد ارسال می‌شد. در هفته سوم، محاسبه هش در سمت ESP32 نیز اضافه شد تا صحت فایل دریافتی پیش از نهایی‌سازی نصب بررسی شود.

برای محاسبه هش، ابتدا زمینه SHA-256 با استفاده از کتابخانه mbedTLS ایجاد و آغاز می‌شود. بخش اصلی این آماده‌سازی در فایل `ota_manager.c` به صورت زیر است:

```
1 mbedtls_md_context_t sha_ctx;
2
3 mbedtls_md_init(&sha_ctx);
4
5 const mbedtls_md_info_t *sha_info =
6     mbedtls_md_info_from_type(
7         MBEDTLS_MD_SHA256
8     );
9
10 mbedtls_md_setup(
11     &sha_ctx,
12     sha_info,
13     0
14 );
15
16 mbedtls_md_starts(
17     &sha_ctx
18 );
```

هنگام دریافت فایل، هر بلوک داده ابتدا وارد محاسبه SHA-256 می‌شود و سپس همان بلوک روی پارتیشن مقصد نوشته می‌شود. در نتیجه، محاسبه هش بدون نیاز به خواندن دوباره فایل و همزمان با فرایند دریافت انجام می‌گیرد:

```
1 mbedtls_md_update(  
2     &sha_ctx,  
3     buffer,  
4     read_len  
5 );  
6  
7 err = esp_ota_write(  
8     ota_handle,  
9     buffer,  
10    read_len  
11 );
```

پس از پایان دریافت، هش نهایی محاسبه و به نمایش هگزادسیمال ۶۴ کاراکتری تبدیل می‌شود:

```
1 unsigned char hash[32];  
2 char calculated_sha256[65];  
3  
4 mbedtls_md_finish(  
5     &sha_ctx,  
6     hash  
7 );  
8  
9 mbedtls_md_free(  
10    &sha_ctx  
11 );  
12  
13 for (int i = 0; i < 32; i++)  
14 {  
15     sprintf(  
16         &calculated_sha256[i * 2],  
17         "%02x",  
18         hash[i]  
19     );  
20 }  
21  
22 calculated_sha256[64] = '\0';
```

در نهایت، مقدار محاسبه شده بدون حساسیت به بزرگی و کوچکی حروف با مقدار دریافتی از سرور مقایسه می شود. در صورت اختلاف، عملیات نوشتن لغو شده و خطای مناسب بازگردانده می شود:

```
1 if (strcasecmp(  
2     calculated_sha256,  
3     expected_sha256) != 0)  
4 {  
5     ESP_LOGE(  
6         TAG,  
7         "SHA-256 verification failed"  
8     );  
9  
10    esp_ota_abort(  
11        ota_handle  
12    );  
13  
14    return ESP_ERR_INVALID_CRC;  
15 }
```

```
I (21353) ota_manager: Calculated SHA-256: f48d7324be922ed99c78a4af34883d405bc2b426a3f0aa5a887b03ebacf601e3  
I (21353) ota_manager: Expected SHA-256: f48d7324be922ed99c78a4af34883d405bc2b426a3f0aa5a887b03ebacf601e3  
I (21363) ota_manager: SHA-256 verification successful
```

شکل ۱: مقایسه مقدار محاسبه شده و مقدار مورد انتظار SHA-256

در صورت برابر بودن دو مقدار، عملیات نصب ادامه پیدا می کند. در غیر این صورت، فرایند OTA متوقف شده و وضعیت شکست برای سرور ارسال می شود.

```
I (17220) ota_manager: Calculated SHA-256: 508f02a879a792f83a92941ba6f331c0eb614798d26294d830dfb51feab179d2  
I (17220) ota_manager: Expected SHA-256: 0000000000000000000000000000000000000000000000000000000000000000  
E (17230) ota_manager: SHA-256 verification failed  
I (17230) api_client: Reporting OTA status: failed  
I (17230) api_client: HTTP request attempt 1/5  
I (17280) api_client: Response status: 200  
I (17280) api_client: Response body: {"update_id":9,"status":"failed"}  
E (17280) main: OTA update failed: ESP_ERR_INVALID_CRC
```

شکل ۲: آزمایش خطای SHA-256 و ثبت وضعیت failed در سرور

۲. فعال سازی و پیاده سازی Rollback

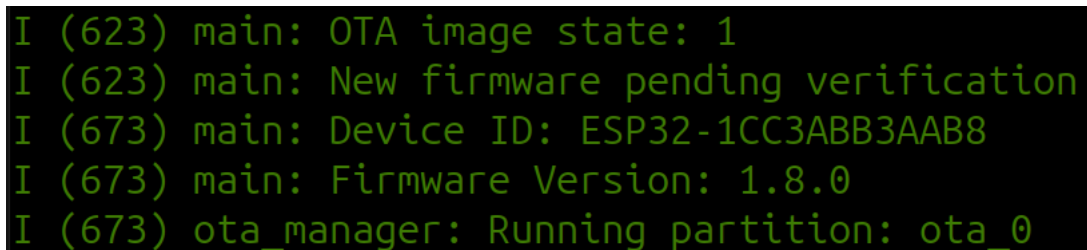
برای جلوگیری از باقی ماندن دستگاه روی یک نسخه معیوب، قابلیت Rollback در تنظیمات ESP-IDF فعال شد.

CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE=y

پس از نصب نسخه جدید، این نسخه ابتدا در وضعیت PENDING_VERIFY قرار می گیرد. اگر آزمون های اولیه برنامه با موفقیت انجام شوند، نسخه معتبر اعلام می شود؛ در غیر این صورت، بوت لودر امکان بازگشت به نسخه قبلی را فراهم می کند.

برای آزمایش مسیر شکست، نسخه 1.29.0 به عنوان نسخه آزمایشی ناموفق در نظر گرفته شد. در این مسیر، در صورت وجود عملیات در انتظار، نتیجه شکست برای سرور ارسال و وضعیت موقت پاک می شود؛ سپس برنامه نسخه فعلی را نامعتبر اعلام کرده و بازگشت و راه اندازی مجدد را درخواست می کند.

```
1 if (strcmp(firmware_version, "1.29.0") == 0)
2 {
3     ESP_LOGE(TAG,
4         "TEST FAILURE: firmware validation failed");
5
6
7     if (ota_state.pending)
8     {
9         if (wifi_connect() == ESP_OK)
10        {
11            api_report_update_status(
12                ota_state.update_id,
13                "failed",
14                "Firmware self-test failed"
15            );
16
17            ota_state_clear();
18        }
19    }
20
21
22    esp_ota_mark_app_invalid_rollback_and_reboot();
23
24    return;
25 }
```



```
I (623) main: OTA image state: 1
I (623) main: New firmware pending verification
I (673) main: Device ID: ESP32-1CC3ABB3AAB8
I (673) main: Firmware Version: 1.8.0
I (673) ota_manager: Running partition: ota_0
```

شکل ۳: تشخیص وضعیت PENDING_VERIFY و درخواست بازگشت به نسخه قبلی

در مسیر عادی، اگر نسخه در حال اجرا با نسخه مقصد ذخیره‌شده مطابقت داشته باشد، برنامه نسخه را معتبر اعلام و امکان Rollback را لغو می‌کند. پس از گزارش موفقیت به سرور نیز وضعیت موقت از NVS پاک می‌شود:

```
1 if (ota_state.pending &&
2     strcmp(firmware_version,
3           ota_state.target_version) == 0)
4 {
5     ESP_LOGI(TAG,
6              "New firmware booted successfully");
7
8
9     esp_err_t valid_err =
10         esp_ota_mark_app_valid_cancel_rollback();
11
12
13     if (valid_err != ESP_OK)
14     {
15         ESP_LOGE(TAG,
16                  "Failed to mark firmware valid: %s",
17                  esp_err_to_name(valid_err));
18     }
19
20
21     if (wifi_connect() == ESP_OK)
22     {
23         if (api_report_update_status(
24             ota_state.update_id,
25             "success",
26             "New firmware booted successfully") == ESP_OK)
27         {
28             ota_state_clear();
29         }
30     }
31
32     return;
33 }
```

۳. آزمایش بازگشت به نسخه سالم قبلی

در آزمایش نهایی، نسخه جدید روی دستگاه نصب شد و در نخستین راه‌اندازی، آزمون اعتبارسنجی برنامه عمداً ناموفق بود. در نتیجه، دستگاه نسخه فعلی را نامعتبر اعلام کرد و فرایند بازگشت آغاز شد.

خروجی سیستم نشان داد که بوت‌لودر به پارتیشن قبلی بازگشته و نسخه سالم دوباره اجرا شده است.

```
W (620) main: Firmware verification failed. Rollback required
W (620) main: Rollback detected
```

شکل ۴: اجرای فرآیند Rollback توسط بوت‌لودر

پس از بازگشت، وضعیت شکست عملیات برای سرور ارسال شد و نتیجه در پنل مدیریتی ثبت گردید.

Registered devices					
Live data from GET /api/devices , refreshed every 10 seconds.					
ID	DEVICE UID	VERSION	MODE	LAST STATUS	LAST SEEN
1	● ESP32-1CC3ABB3AAB8	1.3.0	manual ▾	failed	9/1/2026, 12:34:00 PM

شکل ۵: ثبت نتیجه شکست عملیات بهروزرسانی در پنل

```
W (4140) main: Triggering rollback
I (4500) esp_ota_ops: Rollback to previously worked partition.
```

شکل ۶: اجرای نسخه قبلی پس از Rollback موفق

۴. جلوگیری از ایجاد حلقه بهروزرسانی ناموفق

در آزمایش‌های اولیه مشاهده شد که پس از بازگشت به نسخه قبلی، دستگاه دوباره همان نسخه ناموفق را برای دریافت انتخاب می‌کرد و در نتیجه وارد یک حلقه تکرار می‌شد.

برای رفع این مشکل، پس از تشخیص Rollback، مراحل زیر انجام شد:

- گزارش شکست عملیات به سرور ارسال شد.
- وضعیت موقت ذخیره‌شده در NVS پاک شد.
- ادامه اجرای بررسی نسخه جدید در همان اجرا متوقف شد.

با این تغییر، پس از شکست نسخه جدید، دستگاه فقط یک بار فرایند بازگشت را انجام می‌دهد و در نسخه سالم قبلی باقی می‌ماند. همان‌گونه که در قطعه‌کد مسیر شکست نشان داده شد، پاک‌کردن وضعیت موقت پیش از فراخوانی تابع Rollback و سپس خروج از app_main مانع از ادامه چرخه بهروزرسانی در همان اجرا می‌شود.

۵. جمع‌بندی هفته سوم

خروجی‌های اصلی هفته سوم عبارت‌اند از:

- پیاده‌سازی بررسی صحت فایل ثابت‌افزار با SHA-256 در سمت ESP32.
- تشخیص فایل خراب پیش از نصب و ثبت وضعیت failed.
- فعال‌سازی قابلیت Rollback در ESP-IDF.
- پیاده‌سازی تشخیص وضعیت PENDING_VERIFY.
- بازگردانی خودکار به نسخه قبلی در صورت شکست اجرای نسخه جدید.
- ثبت نتیجه Rollback در سرور.
- جلوگیری از ایجاد حلقه دریافت مجدد نسخه ناموفق.
- آزمایش کامل مسیر دریافت، بررسی صحت، نصب و بازبازی نسخه قبلی.

در پایان هفته سوم، سامانه علاوه بر دریافت و نصب نسخه جدید، توانایی بررسی صحت فایل و بازبازی خودکار پس از شکست به‌روزرسانی را نیز به دست آورد. این مرحله پایداری سامانه را افزایش داد و پیاده‌سازی را به یک سامانه واقعی مدیریت به‌روزرسانی از راه دور نزدیک‌تر کرد.

در هفته آینده، تمرکز اصلی پروژه بر توسعه بخش ارتباطی سامانه و تکمیل فرایند به‌روزرسانی خودکار خواهد بود. در این مرحله، ارتباط فعلی مبتنی بر HTTP به پروتکل MQTT منتقل خواهد شد و با استفاده از کتابخانه Paho MQTT ساختار تبادل پیام بین دستگاه ESP32 و سرور پیاده‌سازی می‌شود. همچنین فرایند به‌روزرسانی خودکار دستگاه بدون نیاز به دخالت دستی توسعه خواهد یافت. در کنار این تغییرات، عملکرد قابلیت‌های پیاده‌سازی‌شده در هفته‌های قبل، شامل بررسی صحت فایل با SHA-256 و مکانیزم Rollback پس از تغییر پروتکل ارتباطی مجدداً آزمایش خواهد شد.